E0123030 / G THARANYA

Implement AND Gate using Single perceptron

```
# ----------------------------------------
# BASIC NEURAL NETWORK IMPLEMENTATION
# AND GATE
# ----------------------------------------

# Step 1: Import required library
import numpy as np

# Step 2: Define input data (AND logic)
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Step 3: Define expected output
y = np.array([[0],
              [0],
              [0],
              [1]])

# Step 4: Initialize weights and bias randomly
weights = np.random.rand(2, 1)
bias = np.random.rand(1)

# Step 5: Define activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Step 6: Define derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Step 7: Training parameters
learning_rate = 0.1
epochs = 10000

# Step 8: Training the neural network
for epoch in range(epochs):

    # Forward propagation
    z = np.dot(X, weights) + bias
    output = sigmoid(z)

    # Calculate error
    error = y - output

    # Backpropagation
    d_output = error * sigmoid_derivative(output)

    # Update weights and bias
    weights += learning_rate * np.dot(X.T, d_output)
    bias += learning_rate * np.sum(d_output)

# Step 9: Display final weights and bias
print("Final Weights:\n", weights)
print("Final Bias:\n", bias)

# Step 10: Apply threshold to get binary output
binary_output = (output >= 0.5).astype(int)

print("\nPredicted Output after Training:")
print(binary_output)
```

```
Final Weights:
 [[5.47902803]
 [5.47902803]]
Final Bias:
 [-8.31123593]

Predicted Output after Training:
[[0]
 [0]
 [0]
 [1]]
```

```
#Implement AND gate using SCIKIT LEARN
from sklearn.linear_model import Perceptron
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([0, 0, 0, 1])

model = Perceptron(max_iter=1000,
                   eta0=0.01,   # changed learning rate
                   random_state=1)

model.fit(X, y)

print("Predicted Output:")
print(model.predict(X))
```

```
Predicted Output:
[0 0 0 1]
```

Implement OR gate using Single Perceptron

00 ⟶ 0, 01 ⟶ 1, 10 ⟶ 1, 11 ⟶ 1

```
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0],
              [1],
              [1],
              [1]])

weights = np.random.rand(2, 1)
bias = np.random.rand(1)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

learning_rate = 0.1
epochs = 10000

for epoch in range(epochs):

    z = np.dot(X, weights) + bias
    output = sigmoid(z)

    error = y - output

    d_output = error * sigmoid_derivative(output)

    weights += learning_rate * np.dot(X.T, d_output)
    bias += learning_rate * np.sum(d_output)

print("Final Weights:\n", weights)
print("Final Bias:\n", bias)

binary_output = (output >= 0.5).astype(int)

print("\nPredicted Output after Training:")
print(binary_output)
```

```
Final Weights:
 [[6.17888447]
 [6.17895932]]
Final Bias:
 [-2.84328756]

Predicted Output after Training:
[[0]
 [1]
 [1]
```

```
 [1]]
```

Implement OR gate using Single Perceptron

00 → 0, 01 → 1, 10 → 1, 11 → 1 using Scikit Learn

```
from sklearn.linear_model import Perceptron
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([0, 1, 1, 1])

model = Perceptron(max_iter=1000,
                   eta0=0.01,
                   random_state=1)

model.fit(X, y)

print("Predicted Output:")
print(model.predict(X))
```

```
Predicted Output:
[0 1 1 1]
```

Implement NAND gate using Single Perceptron 00 → 1, 01 → 1, 10 → 1, 11 → 0

```
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[1],
              [1],
              [1],
              [0]])

weights = np.random.rand(2, 1)
bias = np.random.rand(1)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

learning_rate = 0.1
epochs = 10000

for epoch in range(epochs):

    z = np.dot(X, weights) + bias
    output = sigmoid(z)

    error = y - output

    d_output = error * sigmoid_derivative(output)

    weights += learning_rate * np.dot(X.T, d_output)
    bias += learning_rate * np.sum(d_output)

print("Final Weights:\n", weights)
print("Final Bias:\n", bias)

binary_output = (output >= 0.5).astype(int)

print("\nPredicted Output after Training:")
print(binary_output)
```

```
Final Weights:
 [[-5.47278613]
 [-5.47278614]]
Final Bias:
 [8.30190087]

Predicted Output after Training:
```

```
[[1]
 [1]
 [1]
 [0]]
```

Implement NAND gate using Single Perceptron 00 → 1, 01 → 1, 10 → 1, 11 → 0 using Scikit Learn

```
from sklearn.linear_model import Perceptron
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([1, 1, 1, 0])

model = Perceptron(max_iter=1000,
                   eta0=0.01,
                   random_state=1)

model.fit(X, y)

print("Predicted Output:")
print(model.predict(X))
```

```
Predicted Output:
[1 1 1 0]
```

Implement NOR gate using Single Perceptron 00 → 1, 01 → 0, 10 → 0, 11 → 0

```
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[1],
              [0],
              [0],
              [0]])

weights = np.random.rand(2, 1)
bias = np.random.rand(1)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

learning_rate = 0.1
epochs = 10000

for epoch in range(epochs):

    z = np.dot(X, weights) + bias
    output = sigmoid(z)

    error = y - output

    d_output = error * sigmoid_derivative(output)

    weights += learning_rate * np.dot(X.T, d_output)
    bias += learning_rate * np.sum(d_output)

print("Final Weights:\n", weights)
print("Final Bias:\n", bias)

binary_output = (output >= 0.5).astype(int)

print("\nPredicted Output after Training:")
print(binary_output)
```

```
Final Weights:
 [[-6.17362025]
 [-6.17368923]]
Final Bias:
 [2.84060754]
```

```
Predicted Output after Training:
[[1]
 [0]
 [0]
 [0]]
```

Implement NOR gate using Single Perceptron 00 ⟶ 1, 01 ⟶ 0, 10 ⟶ 0, 11 ⟶ 0 using Scikit Learn

```
from sklearn.linear_model import Perceptron
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([1, 0, 0, 0])

model = Perceptron(max_iter=1000,
                   eta0=0.01,
                   random_state=1)

model.fit(X, y)

print("Predicted Output:")
print(model.predict(X))
```

```
Predicted Output:
[1 0 0 0]
```

Implement NOT gate using Single Perceptron 0 ⟶ 1, 1 ⟶ 0

```
import numpy as np

# Step 2: Define input data (NOT logic)
X = np.array([[0],
              [1]])

# Step 3: Define expected output
y = np.array([[1],
              [0]])

# Step 4: Initialize weights and bias randomly for a single input
weights = np.random.rand(1, 1)
bias = np.random.rand(1)

# Step 5: Define activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Step 6: Define derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Step 7: Training parameters
learning_rate = 0.1
epochs = 10000

# Step 8: Training the neural network
for epoch in range(epochs):

    # Forward propagation
    z = np.dot(X, weights) + bias
    output = sigmoid(z)

    # Calculate error
    error = y - output

    # Backpropagation
    d_output = error * sigmoid_derivative(output)

    # Update weights and bias
    weights += learning_rate * np.dot(X.T, d_output)
    bias += learning_rate * np.sum(d_output)

# Step 9: Display final weights and bias
print("Final Weights:\n", weights)
print("Final Bias:\n", bias)

# Step 10: Apply threshold to get binary output
```

```
binary_output = (output >= 0.5).astype(int)

print("\nPredicted Output after Training:")
print(binary_output)
```

```
Final Weights:
 [[-6.46559872]]
Final Bias:
 [3.124691]

Predicted Output after Training:
[[1]
 [0]]
```

Implement NOT gate using Single Perceptron 0 ⟶ 1, 1 ⟶ 0 using Scikit Learn

```
from sklearn.linear_model import Perceptron
import numpy as np

X = np.array([[0],
              [1]])

y = np.array([1,
              0])

model = Perceptron(max_iter=1000,
                   eta0=0.01,
                   random_state=1)

model.fit(X, y)

print("Predicted Output:")
print(model.predict(X))
```

```
Predicted Output:
[1 0]
```

```
# ---------------------------------------
# BASIC NEURAL NETWORK IMPLEMENTATION
# XOR GATE using Multilayer Perceptron
# ---------------------------------------

# Step 1: Import required library
import numpy as np

# Step 2: Define input data (XOR logic)
# Each row is one input sample
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Step 3: Define expected output
# XOR logic output
y = np.array([[0],
              [1],
              [1],
              [0]])

# Step 4: Initialize weights and bias randomly
# 2 input neurons -> 2 hidden neurons -> 1 output neuron
np.random.seed(1)
W1 = np.random.rand(2, 2)   # Weights from input layer to hidden layer
b1 = np.random.rand(1, 2)   # Bias for hidden layer
W2 = np.random.rand(2, 1)   # Weights from hidden layer to output layer
b2 = np.random.rand(1)      # Bias for output layer

# Step 5: Define activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Step 6: Define derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Step 7: Training parameters
learning_rate = 0.1
epochs = 10000

# Step 8: Training the neural network
for epoch in range(epochs):
```

```
    # -------- Forward Propagation --------
    z1 = np.dot(X, W1) + b1        # Input to hidden layer
    a1 = sigmoid(z1)               # Hidden layer activation

    z2 = np.dot(a1, W2) + b2       # Hidden to output layer
    output = sigmoid(z2)           # Final output

    # -------- Error Calculation --------
    error = y - output

    # -------- Backpropagation --------
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, W2.T) * sigmoid_derivative(a1)

    # -------- Update Weights and Bias --------
    W2 += learning_rate * np.dot(a1.T, d_output)
    b2 += learning_rate * np.sum(d_output)

    W1 += learning_rate * np.dot(X.T, d_hidden)
    b1 += learning_rate * np.sum(d_hidden, axis=0, keepdims=True)

# Step 9: Display final weights and bias
print("Final Weights (Input → Hidden):\n", W1)
print("Final Bias (Hidden):\n", b1)
print("Final Weights (Hidden → Output):\n", W2)
print("Final Bias (Output):\n", b2)

# Step 10: Test the network
print("\nPredicted Output after Training:")
print(output.round())
```

```
Final Weights (Input → Hidden):
 [[3.61819073 5.77402875]
 [3.60479109 5.70602856]]
Final Bias (Hidden):
 [[-5.52792904 -2.37674101]]
Final Weights (Hidden → Output):
 [[-7.94768935]
 [ 7.31108691]]
Final Bias (Output):
 [-3.27790773]

Predicted Output after Training:
[[0.]
 [1.]
 [1.]
 [0.]]
```

```
# ----------------------------------------
# XOR GATE IMPLEMENTATION USING SCIKIT-LEARN
# ----------------------------------------

import numpy as np
from sklearn.neural_network import MLPClassifier

# Input data
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# XOR output
y = np.array([0, 1, 1, 0])

# MLP model
mlp = MLPClassifier(hidden_layer_sizes=(4,),   # increased neurons
                    activation='logistic',     # sigmoid
                    solver='lbfgs',             # stable optimizer
                    max_iter=10000,
                    random_state=1)

# Train
mlp.fit(X, y)

# Predict
predicted_output = mlp.predict(X)

print("Predicted Output after Training:")
print(predicted_output)
```

```
Predicted Output after Training:
[0 1 1 0]
```